Implement depth first search algorithm and Breadth First Search algorithm, Use an undirected graph and develop a recursive algorithm for searching all the vertices of a graph or tree data structure

```
graph = {
   '5': ['3', '7'],
   '3': ['2', '4'],
   '7': ['8'],
   '2': [],
   '4': ['8'],
   '8': []
}

visited = set()   # Set to keep track of visited nodes

# Function for DFS
def dfs(visited, graph, node):
   if node not in visited:
      print(node)
      visited.add(node)

      for neighbour in graph[node]:
         dfs(visited, graph, neighbour)

# Driver Code
print("Following is the Depth-First Search")
dfs(visited, graph, '5')
```

```
# Breadth First Search (BFS)
graph = {
   'A': ['B', 'C', 'D'],
   'B': ['E', 'F'],
   'C': ['G', 'I'],
   'D': ['I'],
   'E': [],
   'F': [],
   'G': [],
   'I': []
}

# Function for BFS
def bfs(visit_complete, graph, current_node):

   visit_complete.append(current_node)

   queue = []
   queue.append(current_node)

   while queue:
      s = queue.pop(0)
      print(s)

      for neighbour in graph[s]:
         if neighbour not in visit_complete:
            visit_complete.append(neighbour)
            queue.append(neighbour)
print("Following is the Breadth-First Search")
bfs([], graph, 'A')
```

```
# DFS and BFS using User Input

graph = {}

# Number of vertices
n = int(input("Enter number of vertices: "))

# Taking graph input
for i in range(n):
    vertex = input("Enter vertex: ")
    neighbours = input(f"Enter neighbours of {vertex} separated by space: ").split()
    graph[vertex] = neighbours

print("\nGraph:", graph)

# ---------- DFS ----------

visited = set()

def dfs(visited, graph, node):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)

        for neighbour in graph[node]:
            dfs(visited, graph, neighbour)

# ---------- BFS ----------

def bfs(graph, start):
    visited = []
```

```
    queue = []

    visited.append(start)
    queue.append(start)

    while queue:
        s = queue.pop(0)
        print(s, end=" ")

        for neighbour in graph[s]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

# Starting node input
start_node = input("\nEnter starting node: ")

# DFS Output
print("\nDepth First Search:")
dfs(visited, graph, start_node)

# BFS Output
print("\n\nBreadth First Search:")
bfs(graph, start_node)
```

Overview:

Depth First Search (DFS) and Breadth First Search (BFS) are graph traversal algorithms used to visit all vertices of a graph or tree.

- DFS visits a node and then explores its neighbouring nodes deeply before backtracking. It uses recursion or stack.
- BFS visits nodes level by level. It uses a queue data structure.

Graph Representation:

The graph is represented using a Python dictionary as an adjacency list.

Example:

```
graph = {
  'A': ['B', 'C'],
  'B': ['D'],
  'C': ['D'],
  'D': []
}
```

Explanation:

- Vertex A is connected to B and C
- Vertex B is connected to D
- Vertex C is connected to D
- Vertex D has no neighbours

DFS Working:

Starting from node A:

1. Visit A
2. Move to B
3. Move to D
4. Backtrack to A
5. Visit C

DFS Output:

A B D C

BFS Working:

Starting from node A:

1. Visit A

2. Visit all neighbours of A → B, C
3. Visit neighbour of B → D

BFS Output:

A B C D

Applications:

- DFS: Path finding, cycle detection, maze solving
- BFS: Shortest path, network traversal, web crawling

Conclusion:

DFS explores deeply while BFS explores level by level. Both algorithms are important for graph and tree traversal in computer science.